PRACTICAL COURSE · BEGINNERS' EDITION

UIscript

Extended Self-Study Guide

A practical guide to the language of interactive interfaces: from your first line of code to complete mini-applications.

19 chapters

4 completed projects

Exercises with solutions

```
окно "Приветствие"
заголовок "Привет, мир!"
текст "Первая программа"
кнопка "Старт"
нажми:
  сообщение "Работает!"
```

Based on the UIscript development environment, version 1.0 (uis_1-0.html), and the original PDF tutorial.

Revised October 2026 · GromAI Studio

How to Use This Book

The best way to learn UIscript is not to read the book straight through without practice, but to alternate between a short explanation, running an example, and changing the code yourself. Open the HTML file of the development environment in your browser. In the editor, replace the text with an example from the book and click “Run” or press Ctrl+Enter (on macOS, usually Cmd+Enter). First get exactly the result described; then change the labels, numbers, colours, and names.

The book uses Russian keywords because they match the default example in the bundled environment. English aliases are also supported, but they do not turn UIscript into JavaScript: the set of commands and expressions remains its own. Keywords in code examples are therefore kept in Russian, while all explanations, labels, and comments are in English. The table below translates the keywords used in this book.

Keyword Glossary

Russian	English	Meaning
окно	window	main panel with a title
заголовок	heading	large heading text
подзаголовок	subheading	second-level heading
текст	text	paragraph or short label
кнопка	button	action button
поле	field	single-line text input
область	text area	multi-line text input
галочка	checkbox	on/off switch
ползунок	slider	selects a number in a range
выбор	selection	drop-down list
вариант	option	item inside a selection
список	list	list of rows that can be managed
картинка	image	image from a URL
видео	video	video player
ссылка	link	link to a resource
разделитель	separator	horizontal line
строка	row	container: elements left to right
столбец	column	container: elements top to bottom
карточка	card	card for grouping content
нажми	click	click event
измени	change	change event
наведи	hover	pointer enters an element

Russian	English	Meaning
уход	leave	pointer leaves an element
сообщение	message	pop-up notification and console entry
показать	show	console entry only
установить	set	set a value
очистить	clear	empty a field, list or variable
добавить	add	add a row to a list
удалить	remove	remove a row from a list
скрыть / открыть / переключить	hide / reveal / toggle	manual visibility
включить / отключить	enable / disable	enable or disable a control
фокус	focus	move the cursor to an element
запрос	request	HTTP request
вызвать	call	call a function as a command
пусть	let	declare a variable
если / иначе	if / else	conditional
повторить	repeat	loop
функция	function	named reusable block
вернуть	return	return a value from a function
каждую секунду	every second	repeating timer
через	after	one-time timer
невидимый	hidden	invisibility property or condition
выключен	disabled	disabled state

Learning Route

Stage	What to master	Result check
1. Basics	Lines, indentation, window, text, button	Make a greeting with a button
2. Data	Element names, variables, expressions	Add two numbers entered by the user
3. Behaviour	Conditions, events, actions	Check a form and show an error
4. Repetition	Loops and functions	Build a multiplication table
5. Dynamics	Timers and reactivity	Make a counter or a clock
6. Applications	Lists, selection, requests, export	Assemble a complete mini-application

Practice Rule

After each section, change at least one line to check that you really understand why the result looks the way it does.

Contents

1 What Is UIscript and How the Environment Works
2 Syntax: Strings, Quotation Marks, Comments and Indentation
3 UI Elements and Nesting
4 Properties and Styling
5 Variables, Values and Expressions
6 Conditions and Input Validation
7 Events and Actions
8 Loops
9 Functions and Scope
10 Timers
11 Reactivity and Conditional Properties
12 Fields, Selectors, Checkboxes and Lists
13 Network and API
14 English Aliases and HTML Export
15 Finding Errors and Debugging
16 Practice: Four Complete Projects
17 Quality Checks Before Publishing
18 Complete Quick Reference
19 Exercises and Solutions
20 Conclusion

CHAPTER 1

What Is UIscript and How the Environment Works

UIscript is a small domain-specific language (DSL) for creating interactive windows and mini-applications in the browser. You describe elements and their behaviour in short lines, and the environment builds the interface. Basic tasks do not require programming directly in HTML, CSS, or JavaScript.

At the same time, UIscript is a separate language with its own syntax. For example, a button is written as `кнопка "Старт"` (button “Start”), and a click handler begins with the line `нажми:` (click:). Not every JavaScript rule applies to UIscript. It has `пусть` (let), `если` (if), `повторить` (repeat), and `функция` (function), but user code has no arbitrary access to all browser capabilities.

1.1. Parts of the Development Environment

Editor is where the program is written. Line numbers appear on the left; below the editor is a status bar showing the cursor position and the number of characters.

Console is a log of messages, warnings, and errors. The command `показать` (show) writes to the console. The command `сообщение` (message) writes there too and also displays a pop-up notification.

Preview shows the result of running the program. The device switch lets you view the interface in desktop or mobile mode.

Examples are ready-made samples for learning the language and page templates. Loading an example replaces the editor's contents; you can undo this with Ctrl+Z.

Help is a short syntax reference. It is useful while you work; this book covers the same topics in more depth.

1.2. Running, Saving and Shortcuts

Action	How to do it
Run the code	“Run” button or Ctrl+Enter (Cmd+Enter on Mac)
Save the source	Ctrl+S, or File → “Save code (.uis)”
Load a source	File → “Import file…”; .uis and .txt are supported
Download the app	File → “Download application (.html)”
Comment out a line	Ctrl+/
Indent	Tab
Remove indentation	Shift+Tab
Close a window or menu	Esc
Undo	Ctrl+Z

The environment saves the editor text to browser storage automatically after a short pause in typing. This is convenient, but it is not a backup. If you switch browsers, clear site data, or run into a problem with local storage, you may lose your code. Save important projects to a separate .uis file.

The option View → “Autostart on input” restarts the program automatically as you edit. For large examples or programs with timers, it is better to run the code manually at first. This makes it clear exactly when the code is executed again.

1.3. First Program

```
окно "Привет"
заголовок "Привет, мир!"
текст "Это моя первая программа на UIscript."
```

1 Paste the example into the editor.

2 Click “Run”.

3 Find the window titled “Hello”.

4 Change the text, run the program again, and compare the results.

The first line creates a window. The two lines with deeper indentation are elements inside the window. You have already used the language's main rule: indentation shows structure.

1.4. First Interactivity

```
окно "Знакомство"
заголовок "Нажми кнопку"
кнопка "Поздороваться"
нажми:
сообщение "Привет! Кнопка сработала."
показать "Это запись в консоли."
```

Commands inside `нажми:` (click:) run when the button is clicked. `сообщение` (message) shows a notification; `показать` (show) only adds an entry to the console. If an event does nothing, check the indentation and make sure the actions are inside the event block.

> **Mini-exercise.** Add a second button, “Secret”, that shows a different message. Don't copy the whole example again. Copy only the button block and change its text and message.

CHAPTER 2

Syntax: Strings, Quotation Marks, Comments and Indentation

2.1. One Line, One Instruction

Usually a line contains one interface element, one property, one event, one action, one variable declaration, or one control structure. Read the code from top to bottom, because order matters. For example, a variable used in a calculation must be declared before that calculation.

```
пусть имя = "Аня"
пусть приветствие = "Привет, " + имя
окно "Программа"
текст приветствие
```

Here `пусть` (let) creates variables. The window displays the value of the variable `приветствие` (greeting).

2.2. Indentation Shows Nesting

Nested lines start to the right of the line they belong to. The convention is two spaces per level. What matters is not the exact number of spaces, but consistency.

```
окно "Анкета"
заголовок "Ваши данные"
строка
поле имя "Имя"
поле город "Город"
кнопка "Готово"
нажми:
сообщение "Привет, {имя}!"
```

The `поле` (field) lines are nested inside `строка` (row), and the row is nested inside `окно` (window). The `сообщение` (message) action is nested inside the `нажми:` (click) event, and the event is nested inside the button.

Compare with an incorrect version:

```
пусть число = 7
число = 8
```

The assignment on the second line is not a child of the variable declaration, so the environment reports an unexpected indent. Commands such as `пусть` (let), assignment, and `показать` (show) do not form containers. An extra indent is allowed only after a construct that actually contains a nested block.

2.3. Quotation Marks and Text

Text strings are usually written in double quotation marks: `"Привет"`. Single quotation marks are also supported in expressions, but the examples in this book use double quotes

consistently.

Simple escape sequences are available inside strings:

```
окно "Строки"
текст "Первая строка\nВторая строка"
текст "Колонка A\tКолонка B"
```

`\n` means a line break and `\t` means a tab. A backslash escapes the following character, for example a quotation mark. If you forget a closing quotation mark, the parser reports an unclosed string.

2.4. Comments

A comment begins with `//`. It is used for explanations and is not executed.

```
// Счётчик хранит число нажатий.
пусть счётчик = 0
окно "Счётчик"
текст "Нажатий: {счётчик}" // текст обновляется автоматически
кнопка "+1"
нажми:
счётчик += 1
```

Inside quotation marks, `//` is ordinary text. For example, the address `"https://example.com"` is not treated as a comment.

2.5. Names of Elements and Variables

Names are identifiers the program uses to refer to elements and variables. Letters, digits, and underscores are allowed; a name must not start with a digit. Cyrillic letters are allowed.

`имя` (name), `Имя` (Name), and `ИМЯ` (NAME) are three different names, so keep to one consistent case.

Good names: `имя`, `итог`, `число_попыток`, `список_покупок`. Bad names: `2числа` (starts with a digit), `цена товара` (contains a space), `если` (a keyword).

Do not use the words `если` (if), `иначе` (else), `пусть` (let), `повторить` (repeat), `функция` (function), or `вернуть` (return) as names for variables or functions. Also avoid the logical constants `да`, `нет`, `истина`, `ложь`, `true`, `false`, and `null`.

A useful habit: a name should be a noun, an action should be a verb, and a comment should explain the reason. For example: `поле имя`, `кнопка "Рассчитать"`, `// Не считаем пустое значение числом`.

> **Mini-exercise.** Write a program with two lines of text and one button. Deliberately remove the indentation from one of the child lines, run the code, find the error, and fix it.

CHAPTER 3

UI Elements and Nesting

An element is a visible part of the application: a heading, a field, a list, a button, or a container. Elements are created on separate lines and can usually take nested properties, events, or child elements.

3.1. Element Reference

Command	Purpose	Example
окно (window)	Main panel with a title	`окно "Профиль"`
заголовок (heading)	Large heading	`заголовок "Добро пожаловать"`
подзаголовок (subheading)	Second-level heading	`подзаголовок "Настройки"`
текст (text)	Paragraph or short label	`текст "Выберите действие"`
кнопка (button)	Action button	`кнопка "Сохранить"`
поле (field)	Single-line text input	`поле имя "Введите имя"`
область (text area)	Multi-line text input	`область заметка "Ваш комментарий"`
галочка (checkbox)	Logical on/off switch	`галочка согласен "Я согласен"`
ползунок (slider)	Selects a number in a range	`ползунок громкость`
выбор (selection)	Drop-down list	`выбор город "Выберите город"`
вариант (option)	Option inside a selection	`вариант "Прага"`
список (list)	List of rows that can be managed	`список задачи`
картинка (image)	Image from a URL	`картинка "https://example.com/a.jpg"`
видео (video)	Video player	`видео "https://example.com/video.mp4"`
ссылка (link)	Link to a resource	`ссылка "Документация"`
разделитель (separator)	Horizontal line	`разделитель`
строка (row)	Container: elements left to right	`строка`
столбец (column)	Container: elements top to bottom	`столбец`
карточка (card)	Card that groups content	`карточка блок "Итоги"`

Important: an image or video needs an accessible URL. If an image does not load, the environment shows a placeholder. Loading network content depends on the availability of the URL and on the site's policies. The video player uses the browser's own controls.

3.2. Element Names: When They Are Needed

You need a name if you want to read a field's value, change text, enable or disable an element, clear it, or change its visibility. For named text elements, the name goes after the keyword and before the text:

```
окно "Приветствие"
поле имя "Как тебя зовут?"
текст результат "Здесь появится приветствие"
кнопка "Показать"
нажми:
установить результат "Привет, {имя}!"
```

Here `имя` (name) is the name of the field, and `результат` (result) is the name of the text element. The label of a field is a hint inside the field, not permanent text.

You can use an input without a name if you never need its value in the program. However, if you plan to use a field in a condition or in a calculation, give it a clear name right away.

3.3. Containers and Visual Structure

`окно` (window), `строка` (row), `столбец` (column), and `карточка` (card) help group elements. In a row, child elements sit side by side; in a column, they sit one below another. A card visually separates its content.

```
окно "Профиль"
заголовок "Карточка пользователя"
строка
  столбец
    текст имя_подпись "Имя"
    поле имя "Алексей"
    текст город_подпись "Город"
    поле город "Бухарест"
  карточка информация "Сводка"
    текст "Данные можно изменить слева."
кнопка "Показать данные"
нажми:
сообщение "{имя}, город: {город}"
```

Avoid putting too many wide fields in one row, because a small screen may not have room for them. For a form with several parameters, a column or several nested rows works better.

3.4. Slider

A slider can have a lower limit, an upper limit, and an initial value. The element's name gives access to its current value.

```
окно "Громкость"
ползунок громкость
  мин 0
  макс 100
  значение 35
текст "Громкость: {громкость}%"
```

If the value is used in arithmetic, check that it falls in the expected range. The slider itself limits the choice to the specified bounds.

3.5. Drop-Down List

Options must be indented inside `выбор` (selection).

```
окно "Город"
выбор город "Выберите город"
  вариант "Бухарест"
  вариант "Клуж"
  вариант "Яссы"
кнопка "Выбрать"
нажми:
если город == ""
  сообщение "Сначала выберите город."
иначе
  сообщение "Вы выбрали город: {город}"
```

If the selection has a prompt, its initial value is an empty string. This lets you check whether the user has made a choice.

3.6. Images, Links and Addresses

An image can be inserted directly:

```
окно "Фото"
картинка "https://picsum.photos/640/360"
  ширина 640
  скругление 12
```

A link or media element can use the `адрес` (address) property. For web links, the environment allows HTTP(S), `mailto:`, `tel:`, and some relative paths. Do not use unknown schemes, and choose external addresses you trust.

```
окно "Полезное"
ссылка "Открыть сайт"
  адрес "https://example.com"
```

> **Mini-exercise.** Create a profile card with a heading, two fields, a city drop-down list, and a button. For now, the button can only show a message with the entered values.

CHAPTER 4

Properties and Styling

Properties configure the appearance of an element that already exists. A property is written indented under the element it belongs to. If a property stands at the top level, the environment cannot determine its owner and shows a warning.

4.1. Full List of Supported Properties

Property	Value	What it changes / where it is useful
фон (background)	Colour	Element background colour
цвет (colour)	Colour	Text colour
размер (size)	Number	Text size in pixels
жирный (bold)	Flag	Bold text
курсив (italic)	Flag	Italic text
центр (centre)	Flag	Centres text
подсказка (hint)	Text	Placeholder for a field or text area, or a tooltip for other elements
ширина (width)	Number or CSS size	Element width
высота (height)	Number or CSS size	Element height
отступ (padding)	Number or CSS size	Inner padding; for rows and columns, also the gap between child elements
скругление (rounding)	Number or CSS size	Corner radius
рамка (border)	Colour or description	Border around the element
тень (shadow)	Flag	Element shadow
прозрачность (opacity)	Number	Transparency, usually from 0 to 1
невидимый (hidden)	Flag	Hides the element
мин (min)	Number	Lower limit of a slider or compatible control
макс (max)	Number	Upper limit of a slider or compatible control
значение (value)	Text or number	Initial value of a field or control, or the text of supported labels
адрес (address)	URL	Address of a link, image, or video
выключен (disabled)	Flag	Blocks a control that supports a disabled state

Numbers in `ширина` (width), `высота` (height), `отступ` (padding), and `скругление` (rounding) are treated as pixels. Examples: `ширина 240`, `высота 80`, `скругление 12`. For colours, use CSS

format: `"#1d4ed8"`, `"#fef3c7"`, `"red"`.

4.2. Flag Values

A flag without a value switches the property on:

```
окно "Акцент"
карточка итог "Итого"
  фон "#eff6ff"
  рамка "#93c5fd"
  скругление 14
  отступ 16
  тень
  текст "Сумма заказа"
    жирный
  текст "125 леев"
    размер 28
    цвет "#1d4ed8"
```

Flags can also take an explicit logical value where this is supported: `жирный` (bold) with no value turns bold on, and `жирный ложь` turns it off. For readability, prefer the plain flag, or the conditional form described in Chapter 11.

4.3. Border and Size

```
окно "Оформление"
карточка примечание "Важно"
  фон "#fff7ed"
  рамка "#fb923c"
  скругление 16
  отступ 18
  текст "Проверьте введённые значения перед расчётом."
    цвет "#9a3412"
  разделитель
  кнопка "Продолжить"
    ширина 220
    высота 44
```

For a border, a colour is usually enough: `рамка "#3b82f6"`. You can also set a full border description, such as `рамка "2px solid #3b82f6"`. A shadow is switched on with the flag `тень` (shadow).

4.4. Not Every Property Suits Every Element

Some properties make sense only on certain elements. `мин` (min), `макс` (max), and `значение` (value) are meant for controls. `адрес` (address) is for links, images, and videos. `выключен` (disabled) is for elements that the browser can block. If you apply a property to an unsuitable element, the command may not stop the program, but the console will show a warning.

> **Mini-exercise.** Style a card in your own way: background, border, rounding, inner padding, a heading, and a button. Then deliberately apply `выключен` (disabled) to a plain text element and read the warning in the console.

CHAPTER 5

Variables, Values and Expressions

A variable is a named place where the program stores a value. Unlike a fixed piece of text, the value of a variable can be recalculated while the program runs.

5.1. Creating, Assigning and Changing

```
пусть очки = 10
очки = очки + 5
очки += 2
окно "Очки"
текст "Итого: {очки}"
```

`пусть очки = 10` (let) creates a variable. `очки = очки + 5` evaluates the right-hand side and writes the result back. The short forms `+=`, `-=`, `*=`, `/=`, and `%=` make in-place changes easier.

You can declare a variable without an initial value: `пусть заметка`. It then holds `null`. The command `очистить заметка` (clear) also empties a variable, setting it to `null`.

Distinguish assignment from comparison: a single `=` assigns a value, while `==` compares two values. Inside a condition, `=` is allowed, but the environment warns that you usually mean `==`.

5.2. Kinds of Values

Kind	Examples	Note
Number	`12`, `3.14`, `-5`	The decimal separator in code is a dot
String	`"Привет"`	Text in quotation marks
Logical value	`истина`, `ложь`	`true` and `false`, `да` and `нет` are also accepted
Empty value	`null`	No value is set

When the user types into a plain text field, the value is a string, even if they entered `12`. Before arithmetic, convert it with `число(имя_поля)` (number(field name)).

5.3. Arithmetic Operators

Operator	Purpose	Example → result
`+`	Addition or joining strings	`2 + 3` → 5
`-`	Subtraction	`9 - 4` → 5
`*`	Multiplication	`6 * 7` → 42
`/`	Division	`7 / 2` → 3.5
`%`	Remainder	`10 % 3` → 1

If one operand of `+` is a string, the values are joined as text. Because of this, `"Сумма: " + 2 + 3` gives the text `Сумма: 23`. For a numerical result, do the arithmetic first: `"Сумма: " + (2 + 3)`.

5.4. Comparisons and Logic

Operators	What they do
==, =	Test for equality
!=	Tests for inequality
>, <, >=, <=	Compare order
и / and / &&	True if both parts are true
или / or / `	
не / not / !	Negates a logical value

Brackets are evaluated first, then multiplication, division, and remainder, then addition and subtraction, and finally comparisons and logical operations. When an expression becomes hard to read, add brackets.

UIscript compares numeric strings by their numeric value when both sides look like numbers: `"10" > "9"` is true. However, equality of strings keeps its textual meaning: `"007" == "7"` is false. An empty string must not be treated as zero; check empty input separately.

5.5. Converting Input to a Number

Incorrect:

```
окно "Сумма"
поле а "Первое число"
поле б "Второе число"
кнопка "Сложить"
нажми:
пусть сумма = а + б
сообщение "Сумма: {сумма}"
```

If the fields contain 2 and 3, the result is `23`, because the program joins the strings.

Correct:

```
окно "Сумма"
поле а "Первое число"
поле б "Второе число"
кнопка "Сложить"
нажми:
пусть сумма = число(а) + число(б)
сообщение "Сумма: {сумма}"
```

The function `число()` (number) understands a decimal comma in user input: `число("1,5")` gives `1.5`. If the text cannot be converted to a number, the result is `NaN` (“not a number”). That is why you should first check the data with the function `число_ли()` (is_number).

5.6. Inserting Values into Text

Curly braces inside a string insert the value of a variable or the result of an expression:

```
пусть имя = "Лена"
пусть цена = 12.5
окно "Чек"
текст "Покупатель: {имя}"
текст "Цена с округлением: {округлить(цена, 1)}"
```

Arithmetic and built-in functions can be used inside the braces. If a name is not defined, the insertion usually becomes an empty fragment, so check the spelling of identifiers.

> **Mini-exercise.** Create a temperature converter: a temperature field, a button, and a result calculated with the formula `(число(температура) * 9 / 5) + 32`. Add `округлить(…, 1)` (round) so that the output does not show too many decimal places.

CHAPTER 6

Conditions and Input Validation

A condition lets the program choose an action based on the data. It is especially useful for forms and calculators, where you must confirm that the user has entered valid values before doing any calculation.

6.1. `если`, `иначе если`, `иначе` (if, else if, else)

```
пусть оценка = 4
если оценка == 5
  сообщение "Отлично!"
иначе если оценка == 4
  сообщение "Хорошо."
иначе если оценка == 3
  сообщение "Удовлетворительно."
иначе
  сообщение "Нужно повторить материал."
```

Checks run from top to bottom. Only the first branch whose condition is true runs. The `иначе` (else) block is optional, but the word `иначе` must line up with its `если` (if). In this environment a colon after `если` is not required, although it is allowed; the examples in this book usually omit it.

6.2. Compound Conditions

```
если число_ли(возраст) и число(возраст) >= 18
  сообщение "Возраст подходит."
иначе
  сообщение "Введите возраст от 18 лет."
```

`и` (and) requires both parts to be true. `или` (or) requires only one part to be true. `не` (not) negates a value. The environment also supports `and`, `or`, `not`, `&&`, `||`, and `!`.

Expressions with `и` and `или` are evaluated with short-circuiting: if the first part already determines the result, the second part may not be evaluated. This lets you write `пусто(имя) или длина(имя) < 2`, because when the name is empty the length check is never needed.

6.3. Truthiness of a Value

A condition can use a value directly, not only a comparison. Non-empty values other than `0`, `ложь` (false), and `null` are normally treated as true. A checkbox has a logical value, so it is natural to write `если согласен` (if agreed).

Warning: the string `"0"` is non-empty text, and as a string it counts as true. If you are checking a number, convert it first: `если число(ввод) != 0`.

6.4. Four Useful Validation Functions

Function	What it does	Example
`число_ли(x)` (is_number)	Checks whether the value reads as a finite number	`число_ли("12,5")` → true

Function	What it does	Example
`пусто(x)` (is_empty)	Checks for no value; treats an empty string, `null`, `ложь`, and an empty array as empty	`пусто("")` → true
`обрезать(x)` (trim)	Removes spaces from both ends of a string	`обрезать(" да ")` → `"да"`
`длина(x)` (length)	Returns the length of a string or array	`длина("привет")` → 6

6.5. Checking a Name

```
окно "Регистрация"
поле имя "Ваше имя"
галочка согласен "Принимаю условия"
кнопка "Продолжить"
нажми:
если обрезать(имя) == ""
  сообщение "Введите имя."
  фокус имя
иначе если не согласен
  сообщение "Подтвердите согласие."
иначе
  сообщение "Добро пожаловать, {обрезать(имя)}!"
```

`обрезать(имя) == ""` catches not only an empty field but also a field containing only spaces. `фокус имя` (focus) moves the cursor back to the right field.

6.6. Safe Numeric Calculation

```
окно "Квадрат числа"
поле ввод "Введите число, например 7"
текст результат "Результат появится здесь"
кнопка "Посчитать"
нажми:
если не число_ли(ввод)
  сообщение "Введите корректное число."
  фокус ввод
иначе
  пусть x = число(ввод)
  установить результат "Квадрат: {округлить(x * x, 2)}"
```

The main sequence is: check first, then convert, then calculate, then output. Do not multiply `число(ввод)` by itself until you have checked `число_ли(ввод)`, or else `NaN` can propagate into later calculations.

> **Mini-exercise.** Extend the name check so that names shorter than two characters are rejected. Then add a check that the amount in the field `сумма` is a positive number.

CHAPTER 7

Events and Actions

An event is a moment when the user interacts with an element. Actions inside an event block run in response to that interaction.

7.1. Events

Event	When it fires	Example use
`нажми:` (click)	A click or tap on an element	Submitting a form
`измени:` (change)	A field, text area, selection, checkbox, or slider value changes	Recalculating while typing
`наведи:` (hover)	The pointer enters an element	A tooltip on hover
`уход:` (leave)	The pointer leaves an element	Removing a highlight after hover

Important: the `наведи:` (hover) and `уход:` (leave) events depend on a mouse pointer. On a touch screen, the user may not have a hover state. Do not make hover the only way to access an important feature.

7.2. Actions

Command	Purpose
`сообщение "…"` (message)	Pop-up notification and console entry
`показать "…"` (show)	Console entry only
`установить имя значение` (set)	Sets a value on an element or variable
`очистить имя` (clear)	Empties a field, list, or text; sets a variable to `null`
`добавить список значение` (add)	Adds a row to a named list
`удалить список индекс` (remove)	Removes a list item by index
`скрыть имя` (hide)	Hides a named element
`открыть имя` (reveal)	Shows an element that was hidden manually
`переключить имя` (toggle)	Toggles an element's manual visibility
`включить имя` / `отключить имя` (enable / disable)	Enables or blocks a suitable control
`фокус имя` (focus)	Moves focus to an element, such as a text field
`запрос … в переменная` (request)	Sends an HTTP(S) request; see Chapter 13
`вызвать функция(…)` (call)	Calls your own function when its result is not needed as an expression

7.3. Why `сообщение` and `показать` Are Different

```
окно "Журнал"
кнопка "Проверить"
нажми:
показать "Событие зарегистрировано"
сообщение "Пользователь нажал кнопку"
```

You find the first entry in the console. The second also appears in the console, but it is additionally shown to the user as a notification.

7.4. Setting Values

```
окно "Статус"
текст статус "Ожидание"
кнопка "Завершить"
нажми:
установить статус "Готово"
установить прогресс 100
показать "Статус изменён"
```

If the name after `установить` (set) matches an element's name, the command changes that element's value. If it matches a variable's name, the command writes to the variable. For readability, avoid giving a variable and an element the same name, because that can make access to the element ambiguous.

An important detail: an explicit `установить` (set) on the text of a reactive element replaces its computed content and turns off the automatic updates of that text. If a label should always reflect a variable, use a string with curly braces and change the variable rather than setting the text by hand. See Chapter 11.

7.5. Hiding, Showing and Focus

```
окно "Настройки"
текст расширенные "Дополнительные настройки"
поле ключ "Введите код"
кнопка "Показать настройки"
нажми:
переключить расширенные
кнопка "Очистить код"
нажми:
очистить ключ
фокус ключ
```

`скрыть` (hide), `открыть` (reveal), and `переключить` (toggle) change the manual visibility. They do not remove the element from the program, and a hidden element keeps its value. `отключить` (disable) blocks interaction if the element supports the disabled state; `включить` (enable) removes the block.

> **Mini-exercise.** Create a “Show details” toggle. When clicked, it hides and reveals a card without deleting the card's contents.

CHAPTER 8

Loops

A loop repeats a block of code a fixed number of times. UIscript uses the construct `повторить N раз:` (repeat N times:). It is suitable for tables, sequences of buttons, lists, and small repeated calculations.

8.1. Sum of a Sequence

```
пусть сумма = 0
повторить 100 раз:
  сумма += i
окно "Сумма"
заголовок "Сумма от 1 до 100"
текст "Результат: {сумма}"
```

Inside the loop, the variable `i` takes the values from 1 to N. In this example the sum is 5050. A loop with zero repetitions does nothing. The number of repetitions can be an expression, but it must evaluate to a number.

8.2. `i` and `индекс` (index)

Pass	i	индекс
First	1	0
Second	2	1
Tenth	10	9

`i` is convenient for human-friendly numbering; `индекс` (index) is used when you need counting from zero. In nested loops, the inner loop temporarily uses its own `i` and `индекс`, so if you need the outer number, save it in a separate variable before starting the inner loop.

8.3. Multiplication Table

```
окно "Таблица умножения"
поле множитель "Введите число"
кнопка "Построить"
нажми:
если не число_ли(множитель)
  сообщение "Введите число, например 7."
иначе
  пусть число_таблицы = число(множитель)
  очистить строки
  повторить 10 раз:
    добавить строки "{число_таблицы} × {i} = {округлить(число_таблицы * i, 4)}"
  список строки
```

Here the loop does more than count: it builds the content of a list. `очистить строки` (clear) empties the previous result, so that the table does not double after every click.

8.4. Generating a Row of Elements

```
окно "Динамические кнопки"
строка
  повторить 5 раз:
    кнопка "Кнопка {i}"
      нажми:
        сообщение "Номер кнопки: {i}"
```

A loop can create elements inside a window or container. When an element is created, the environment saves a snapshot of the local values (`i`, `индекс`, and function parameters). The handler then sees that snapshot, not the current value of the variable. This is a good way to generate repeated interfaces without copying identical lines.

8.5. Limits of Loops

- A single loop allows at most 10,000 repetitions.
- A non-numeric repetition count causes an error.
- UIscript has no `while`, `break`, or `continue`.
- To stop early when a condition is met, move the loop into a function and use `вернуть` (return) when the result is found.

> **Mini-exercise.** Create a task generator: the user enters a count, clicks a button, and the program adds rows “Task 1”, “Task 2”, and so on to a list. Check that the count is a valid number and does not exceed a reasonable limit for the interface.

CHAPTER 9

Functions and Scope

A function is a named block of code that can be reused. It takes parameters, performs actions, and can return a result.

9.1. Declaring and Calling

```
функция квадрат(x):
  вернуть x * x
пусть результат = квадрат(7)
окно "Функция"
текст "Квадрат 7: {результат}"
```

`функция квадрат(x):` (function square(x):) describes a function with the parameter `x`. `вернуть` (return) ends the function and hands the value back to the place where the function was called. If a function returns nothing, the result of the call is `undefined` (an empty value).

If you don't need a result and want to call the function as a separate command, use `вызвать` (call):

```
функция привет(имя):
  сообщение "Здравствуйте, {имя}!"

окно "Приветствие"
кнопка "Сказать привет"
нажми:
  вызвать привет("Мир")
```

9.2. Parameters and Local Variables

```
пусть x = 1
функция увеличить(x):
  пусть временно = x + 1
  вернуть временно

окно "Локальные значения"
текст "Результат: {увеличить(10)}"
текст "Внешняя x: {x}"
```

Parameters, and variables declared inside a function with `пусть` (let), are local to that call. In this example, the outer variable `x` stays equal to 1. Do not expect a variable that is first created inside a function to remain available outside it.

Functions can, however, access global variables and change them when needed:

```
пусть очки = 0
функция добавить_очки(количество):
  очки += количество

окно "Очки"
текст "Всего: {очки}"
кнопка "+5"
нажми:
  вызвать добавить_очки(5)
```

9.3. Early Return

`вернуть` (return) ends the function immediately. This is also useful inside loops:

```
функция первое_число_больше(n):
  повторить 100 раз:
    если i > n
      вернуть i
  вернуть 0

окно "Поиск"
текст "Для 7: {первое_число_больше(7)}"
```

Because `вернуть` exits the function, the loop stops along with it. Add an exit condition at the end of the function so that every valid input gives a clear result.

9.4. Recursion

A function can call itself. This is called recursion. It must always have a termination condition:

```
функция факториал(n):
  если n <= 1
    вернуть 1
  вернуть n * факториал(n - 1)

окно "Факториал"
текст "5! = {факториал(5)}"
```

In this example 5! equals 120. The environment limits the depth of nested calls to 100. For tasks that are simpler to solve with a loop, a loop is usually clearer and safer.

9.5. Common Mistakes with Functions

- You can call a function directly as an expression: `квадрат(5)`. But if the call is a separate command, write `вызвать квадрат(5)`.
- `вернуть` (return) is allowed only inside a function.
- Do not name your function after a built-in, such as `длина` (length) or `число` (number).
- Watch the brackets and commas in the parameter list.
- Do not create infinite recursion: every path must reach a base case.

Mini-exercise. Write a function `скидка(цена, процент)` (discount(price, percent)) that returns the price after the discount. Call it for several pairs of numbers and display the results using `округлить` (round).

CHAPTER 10

Timers

Timers run actions based on time. A repeating timer starts after a given interval; a one-shot timer runs once after a delay.

10.1. Types of Timers

Syntax	Behaviour
`каждую секунду:` (every second:)	Runs the block roughly once per second
`каждые 5 сек:` (every 5 sec:)	Repeats the block every 5 seconds
`каждые 500 мс:` (every 500 ms:)	Repeats the block every 500 milliseconds
`через 3 сек:` (after 3 sec:)	Runs the block once, after about 3 seconds

This version recognises second units (`сек` and its longer forms with the word “`секунд`”) and millisecond units (`мс` and its longer forms), as well as English equivalents. In teaching code, stick to the short forms `сек` and `мс`. The interval must be a positive number. A repeating timer has a minimum interval of 50 ms, and at most 50 timers can run at the same time.

10.2. Seconds Counter

```
пусть тики = 0
окно "Таймер"
заголовок "Прошло секунд: {тики}"
каждую секунду:
  тики += 1
```

Each tick changes the variable. The heading updates thanks to reactivity: the timer does not need to set the heading's text manually.

10.3. Clock

```
окно "Часы"
заголовок "{сейчас()}"
  центр
  размер 44
текст "Сегодня: {дата()}"
  центр
каждую секунду:
```

The functions `сейчас()` (now) and `дата()` (date) return the time and date from the device clock. A timer with no actions is a legitimate technique: it makes reactive labels recalculate periodically, even when no program variables change.

10.4. One-Time Delay

```
окно "Напоминание"
кнопка "Напомнить через 3 секунды"
нажми:
сообщение "Таймер запущен."
через 3 сек:
  сообщение "Три секунды прошли!"
```

The body of `через 3 сек:` (after 3 sec:) runs once. When the whole program is run again, timers from the previous run are stopped and new ones are created. Timers work only while the page running the program is open.

Be careful with frequent timers. Do not update the interface more often than necessary. `каждые 500 мс:` (every 500 ms:) is fine for a simple effect, but a frequent timer with expensive calculations can make the page less responsive. For learning counters, `каждую секунду:` (every second:) is usually enough.

> **Mini-exercise.** Build a countdown with a variable “remaining”. When “Start” is pressed, decrease it once per second. When it reaches zero, show a message and stop decreasing, using a logical variable “running”.

CHAPTER 11

Reactivity and Conditional Properties

Reactivity means that displayed text updates by itself when the data it depends on changes. In the simple case, it is enough to put a variable name in curly braces.

11.1. Reactive Text

```
пусть счётчик = 0
окно "Счётчик"
заголовок "{счётчик}"
  центр
  размер 48
строка
  кнопка "−"
  нажми:
    счётчик -= 1
  кнопка "Сброс"
  нажми:
    счётчик = 0
  кнопка "+"
  нажми:
    счётчик += 1
```

The heading updates after the variable is assigned. You do not need the command `установить заголовок` (set heading) for this; in fact, setting the text manually replaces the reactive formula, and the label stops following the variable.

Reactive elements can be a window or card with a dynamic title, a heading, a subheading, text, a button, a link, and a checkbox label. Field values can also be used inside insertions, so the interface can change its text while the user types.

11.2. Reactive Calculation

```
окно "Цена"
поле цена "Сумма заказа"
ползунок скидка
  мин 0
  макс 50
  значение 10
текст "Скидка: {скидка}%"
текст "К оплате: {округлить(число(цена) * (1 - скидка / 100), 2)}"
```

When the user changes the price field or moves the slider, the label recalculates. A reactive formula must be an expression that can be evaluated. Use the check `число_ли(цена)` (is_number) if an empty input must not produce `NaN`.

11.3. Conditional Properties

In this implementation, the conditional form of a property (`если`) is supported for five properties: `выключен` (disabled), `невидимый` (hidden), `жирный` (bold), `курсив` (italic), and `центр` (centre).

```
пусть попытки = 0
окно "Лимит"
текст предупреждение "Лимит достигнут"
  цвет "#dc2626"
  жирный если попытки >= 5
  невидимый если попытки < 5
кнопка "Ещё попытка"
нажми:
  попытки += 1
```

You cannot write `цвет если …` (colour if …) or `фон если …` (background if …). For dynamic styling through a condition, only the flags listed above are supported. To choose a colour, you can use two labels with different colours and control their visibility with conditions.

11.4. Manual and Conditional Visibility

An element can have a manual visibility state (`скрыть` / `открыть`, hide / reveal) and a condition `невидимый если …` (hidden if …). If you need simple conditional showing or hiding, control it with the condition. If the user toggles a panel manually, use the actions `скрыть`, `открыть`, and `переключить`. Do not mix both approaches without a reason, because the behaviour becomes harder to explain.

> **Mini-exercise.** Create an amount field and two labels: “Enter an amount” is visible when the input is empty or not a number, and the total is visible only when the input is a valid number. Use `невидимый если` (hidden if) and `число_ли` (is_number).

CHAPTER 12

Fields, Selectors, Checkboxes and Lists

Input controls turn a static page into an application. Every named control has a value that you can read by its name.

12.1. Single-Line and Multi-Line Fields

```
окно "Заметка"
поле тема "Тема заметки"
область текст_заметки "Напишите текст заметки"
кнопка "Показать"
нажми:
  если пусто(обрезать(тема))
    сообщение "Введите тему."
  иначе
    сообщение "Тема: {обрезать(тема)}; заметка: {текст_заметки}"
```

`поле` (field) is for a single line; `область` (text area) is for several lines. Both return the entered text when read. Convert the value to a number only where arithmetic is actually needed.

12.2. Checkbox

A checkbox returns a logical value: `истина` (true) if it is ticked and `ложь` (false) if not.

```
окно "Подтверждение"
галочка согласен "Я согласен с условиями"
кнопка "Продолжить"
нажми:
  если согласен
    сообщение "Продолжаем."
  иначе
    сообщение "Сначала отметьте галочку."
```

You can change the state with `установить согласен истина` or `установить согласен ложь`. `очистить согласен` (clear) removes the tick. Check logical flags directly, without comparing them to `истина`, unless the expression becomes less clear.

12.3. Drop-Down Selection

The value of a `выбор` (selection) is the text of the chosen option. If nothing is chosen and there is a prompt, the value is empty.

```
окно "Заказ"
выбор размер "Выберите размер"
  вариант "Маленький"
  вариант "Средний"
  вариант "Большой"
галочка доставка "Нужна доставка"
кнопка "Оформить"
нажми:
  если размер == ""
    сообщение "Выберите размер."
  иначе если доставка
    сообщение "Заказ: {размер}, с доставкой."
  иначе
    сообщение "Заказ: {размер}, самовывоз."
```

12.4. List of Rows

A `список` (list) differs from a `выбор` (selection): a list lets you add and remove rows from the program, and its value can be treated as an array of text items.

```
окно "Список покупок"
поле товар "Что купить?"
кнопка "Добавить"
нажми:
  если пусто(обрезать(товар))
    сообщение "Введите название товара."
  иначе
    добавить покупки "{обрезать(товар)}"
    очистить товар
    фокус товар
список покупки
текст "Всего позиций: {длина(покупки)}"
строка
  кнопка "Удалить первый"
  нажми:
    удалить покупки 0
  кнопка "Удалить последний"
  нажми:
    удалить покупки -1
  кнопка "Очистить всё"
  нажми:
    очистить покупки
```

`добавить покупки …` (add) appends a new row. `удалить покупки 0` (remove) removes the first row, and `удалить покупки -1` removes the last one. `очистить покупки` (clear) empties the list completely. Indexing for removal starts at zero; the negative index `-1` refers to the last item.

12.5. Setting and Clearing Values

- `установить город "Прага"` (set) selects the option “Прага” if it exists in the list.
- `установить доставка истина` ticks the checkbox.
- `установить товар "Молоко"` sets the text of the field.
- `очистить товар` (clear) empties the field.
- `очистить покупки` (clear) empties the list.

`установить` (set) is for programmatic control of a value, and `очистить` (clear) is for resetting it. Remember that an input field and a variable are different things: a field is read by its element name, but in arithmetic it usually requires `число(поле)` (number(field)).

> **Mini-exercise.** Create a to-do list with a field, an add button, a count of tasks, a button that removes the last row, and a button that clears everything. Additional condition: empty strings and strings containing only spaces must not be added.

CHAPTER 13

Network and API

The `запрос` (request) command retrieves data from a server or sends data to it. The request runs asynchronously: the program keeps working, and the target variable receives the result when the request completes.

13.1. Basic GET Request

```
пусть ответ = "Ожидание"
окно "Данные сервера"
текст "Ответ: {ответ}"
кнопка "Загрузить"
нажми:
  ответ = "Загрузка…"
  запрос "https://api.example.com/message" в ответ
```

`api.example.com` in this example is a placeholder address, not a real data source. Replace it with the URL of a real API that returns a response and allows requests from your page. If the response is JSON, the variable usually holds an object; if the server returns plain text, the value is a string.

13.2. Accessing Nested Data: `путь()` (path)

The function `путь(объект, "a.b[0]")` (path(object, "a.b[0]")) reads a nested property from an object and can access array elements by index in square brackets.

Suppose the server returns JSON like this:

```
{
  "user": { "name": "Mira" },
  "items": ["Книга", "Ручка"]
}
```

Then the expressions `путь(ответ, "user.name")` and `путь(ответ, "items[0]")` are useful. A field exists only if the server actually sent it; a missing field gives an empty or undefined value.

13.3. Method and Request Body

The action accepts optional method and body parameters, for example `запрос "URL" метод "POST" тело "текст" в ответ` (request "URL" method "POST" body "text" into response). The default method is GET. The internal mechanism sends a request body for methods other than GET and HEAD; an object body is encoded as JSON. The language syntax provides no separate command for arbitrary HTTP headers, so an API that requires special headers or complex authorisation may be inconvenient to call directly from UIscript.

13.4. Limits of Network Requests

- Only `http://` and `https://` URLs are allowed.
- The browser enforces CORS policy. If the server does not allow requests from your origin, the environment cannot bypass that restriction.

- The request timeout in this implementation is about 15 seconds.
- Not more than 10 requests may run at the same time.
- If an HTTP or network error occurs, the target variable may receive `null`; look for details in the console.
- If the server requires an API key, do not put a secret key in a public page that you distribute.

13.5. How to Debug a Request

1. Check that the URL opens and that the server supports the method you are using.
2. Open the browser developer console (F12 on desktop browsers) if you need details about a network error.
3. Check the response type: is it a string or a JSON object?
4. For JSON, start by displaying the whole response, then use `путь()` (path) to reach the field you need.
5. If the message mentions CORS, fix the configuration of the server or API, not the expression in UIscript.

> **Mini-exercise.** Find a public API that explicitly allows browser requests without a secret key. First display the entire response, then extract a single field with `путь()` (path). Use only a service you trust.

CHAPTER 14

English Aliases and HTML Export

14.1. English Versions of Commands

The environment supports a number of English aliases for commands, elements, properties, and events. For example, `window` corresponds to `окно`, `text` to `текст`, `button` to `кнопка`, `click` to `нажми`, `if` to `если`, `let` to `пусть`, `function` to `функция`, `return` to `вернуть`, `show` to `показать`, `message` to `сообщение`, and `background` to `фон`.

```
window "Greeting"
  heading "Hello!"
  button "Say hello"
    click:
      message "Hello from UIscript!"
```

It is best to choose one style and stick to it. In this book's Russian code, use Russian commands; use English aliases when you are reading a ready-made English example. Labels in quotation marks are not translated automatically.

14.2. Links and Sharing Code

The environment includes functions for opening and importing sources, saving .uis files, creating links for sharing code, and exporting a finished HTML page. When you share code through a link, make sure the link contains the current version of the program. Long programs are easier to share as a separate .uis file.

14.3. Exporting a Standalone HTML Application

To produce a page that runs without the development environment:

1. Run the source and fix any errors.
2. Save the .uis file as a source copy.
3. Choose File → “Download application (.html)”.
4. Open the downloaded file in a browser and test the main scenarios again.
5. Before sending it to someone else, test the page in the same browser and on the same device where it will be opened.

The exporter builds a separate HTML page containing the program's source code and the required built-in runtime. Before saving, the environment checks that the source parses, performs a dry run without starting timers or network requests, and then checks that the embedded JavaScript builds. This reduces the risk of an export error, but it does not replace testing the file after download.

14.4. What Standalone Means, and What It Does Not

An exported HTML page does not need the UIscript editor in order to run its interface. However, this does not make external resources local: images, videos, and API requests by URL still depend on network availability, CORS rules, and the server. If the page will be used without internet access, external URLs may not work.

Security. A standalone HTML file contains executable code. Send it only when you are ready to reveal the contents of the application. Do not store passwords, secret keys, or private data in it.

14.5. What to Do If the HTML Does Not Download

1. Check that the editor is not empty.
2. Click “Run” and fix any syntax or runtime errors.
3. Repeat the export and read the message in the console.
4. If the browser blocks the download, try a different browser, or use a window with the HTML code if the environment offers one, and save its contents to a file with the .html extension.
5. Open the saved page as a separate file. Testing only in the preview is not enough.

> **Mini-exercise.** Export one of the small projects from Chapter 16 and check three things: the interface displays, the buttons work, and the dynamic labels update. Then turn off your network connection and see what happens to the external images.

CHAPTER 15

Finding Errors and Debugging

An error is not a reason to rewrite the program from scratch. Read the error message, go to the line it points to, and check that line together with the lines next to it. The cause is often a wrong indent, a wrong name, or a value that came from an input field.

15.1. Three Categories of Problems

- **Parse error:** the language cannot read the program text, so the program does not start.
- **Runtime error:** the syntax is accepted, but a problem occurs when an action runs.
- **Warning:** the program may continue running, but its behaviour probably does not match what you intended.

In this environment, an error message usually includes a line number. Clicking the error line in the console or the error notification helps you jump to the place where the error appeared.

15.2. Common Parse Errors

Message / symptom	Why it happens	How to fix it
“Unexpected indent”	A command is indented too far, or a line is not inside a suitable block	Align neighbouring commands; use a child indent only inside an element or block
“`иначе` cannot appear without `если`”	`иначе` (else) does not line up with its preceding `если` (if), or the condition is missing	Check the vertical alignment of `если`, `иначе если`, and `иначе`
“Unrecognised construct”	A typo in a keyword, or an unsupported command	Compare the word with the quick reference
“Unclosed string”	A closing quotation mark is missing	Close the text with a quotation mark of the same type
“Invalid timer syntax”	An unsupported time unit, or an incomplete construct	Use `каждую секунду:`, `каждые 5 сек:`, or `через 2 сек:`
“Colour cannot be used with `если`”	The colour property has a condition	Remove the condition, or apply it only to `невидимый`, `жирный`, `курсив`, `центр`, or `выключен`

Example of an incorrect structure:

```
окно "Пример"
если истина
текст "Внутри условия"
иначе
  текст "Отступ отличается"
```

Corrected version:

```
окно "Пример"
если истина
  текст "Внутри условия"
иначе
  текст "Отступ отличается"
```

15.3. Common Runtime Errors

Message / symptom	Typical cause	What to check
“The repeat count must be a number”	The field is empty or contains non-numeric text	Check with `число_ли()` (is_number) before `повторить` (repeat)
“Too many repetitions”	The loop requests more than 10,000 passes	Limit the user's input
“The timer time must be a positive number”	The time is zero, negative, or not a number	Check the input and the interval formula
“Unknown function”	A typo, a missing function, or a name mismatch	Check the declaration and the brackets
“`вернуть` can only be used inside a function”	`вернуть` (return) is placed outside a function	Move the command inside `функция` (function)
“`вариант` can only be written inside `выбор`”	`вариант` (option) is at the wrong indent level	Nest it inside `выбор` (selection)
“List not found”	An `добавить` / `удалить` command refers to a different identifier	Compare the list name character by character
“Only http(s) addresses are allowed”	The request uses an unsupported URL scheme	Use `https://` or `http://`
Recursion too deep	A function calls itself without reaching its end	Fix the base case, or use a loop

15.4. Warnings You Should Not Ignore

- **The name is not defined.** Check for a typo, or for a variable that was never declared with `пусть` (let).
- **A variable shadows an element.** Do not give a variable the same name as a field, text element, or button: references by that name become ambiguous for the user and can make the element inaccessible.
- **A duplicate element name.** Give different elements unique identifiers if you plan to refer to them.
- **A property is not supported by the element.** Move the property to an element of a suitable type.
- **A property stands outside its element.** Add the correct indent under the intended element.

15.5. Logical Errors with No Message

These errors are harder to notice: the program runs, but the result is wrong.

Strings are joined instead of added. If `а` and `б` are input fields, `а + б` may join the text. The fix is `число(а) + число(б)` after checking with `число_ли`.

The value does not update. If a text element contained `{очки}` and you then used `установить текст_очков "…"` (set), the replacement disabled the reactive formula. Remove the manual `установить` and update the variable `очки` instead.

The list keeps growing. If you do not clear the old list before generating a new result, every re-run adds duplicates. Run `очистить имя_списка` (clear) before the loop.

The index is wrong. Lists are indexed from zero: `0` is the first item and `-1` is the last one when removing. Do not confuse `і` (counting from one) with `индекс` (counting from zero).

15.6. A General Debugging Algorithm

1. **Reproduce the error.** Save a copy of the code and run it again.
2. **Locate the place.** Start with the line the console points to, then read the lines above and below it.
3. **Check the structure.** Indentation, the colon after blocks, quotation marks, brackets, names.
4. **Check the input data.** Empty string, spaces, non-numeric characters, an unselected option.
5. **Narrow the example.** Temporarily remove unnecessary elements until the smallest fragment that still shows the problem remains.
6. **Add `показать` (show).** Print intermediate values: `показать "x={x}"`.
7. **Restore the parts one at a time.** Once a small example works, add back one block at a time.

Do not fix five different problems at once, because then it is hard to tell which change helped.

CHAPTER 16

Practice: Four Complete Projects

Each project is built from the commands described above. First paste the entire code, then run it and carry out the check steps. All lines can be edited: the goal is not only to get a nice interface, but to understand what each construct does.

Project 1. Four-Operation Calculator

What we study: input fields, a drop-down selection, validation, branching, and rounding.

```
окно "Калькулятор"
заголовок "Четыре операции"
строка
  поле первое "Число A"
  выбор операция "Операция"
    вариант "+"
    вариант "-"
    вариант "*"
    вариант "/"
  поле второе "Число B"
текст итог "Введите числа и нажмите «Посчитать»."
кнопка "Посчитать"
нажми:
  если не число_ли(первое) или не число_ли(второе)
    установить итог "Ошибка: введите оба числа."
  иначе
    пусть a = число(первое)
    пусть b = число(второе)
    если операция == ""
      установить итог "Выберите операцию."
    иначе если операция == "+"
      установить итог "Результат: {округлить(a + b, 6)}"
    иначе если операция == "-"
      установить итог "Результат: {округлить(a - b, 6)}"
    иначе если операция == "*"
      установить итог "Результат: {округлить(a * b, 6)}"
    иначе если b == 0
      установить итог "На ноль делить нельзя."
    иначе
      установить итог "Результат: {округлить(a / b, 6)}"
```

Check: 12 + 3 gives 15; 12 / 4 gives 3; an empty field gives a clear error message; attempting to divide by zero produces no calculation.

Improve it yourself: add a button that resets the fields; show which operation the user selected; limit the number of decimal places.

Project 2. Shopping List

What we study: adding, removing, clearing, focus, validation, and a reactive count of items.

```
окно "Покупки"
заголовок "Список покупок"
строка
  поле товар "Что нужно купить?"
  кнопка "Добавить"
    нажми:
      если пусто(обрезать(товар))
        сообщение "Введите название товара."
        фокус товар
      иначе
        добавить покупки "{обрезать(товар)}"
        очистить товар
        фокус товар
текст "Количество позиций: {длина(покупки)}"
список покупки
строка
  кнопка "Удалить первый"
  нажми:
    удалить покупки 0
  кнопка "Удалить последний"
  нажми:
    удалить покупки -1
  кнопка "Очистить список"
  нажми:
    очистить покупки
```

Check: an empty string is not added; the field is cleared after an item is added; the count of items changes; the remove and clear commands work.

Why `длина(покупки)` is used: a named list is read as an array of items. The function `длина()` (length) returns the number of array elements.

Improve it yourself: add a separate “Delete all” button with a confirmation step, or disable the removal buttons when the list is empty. Think about which check is needed so that the user does not see an error message without a reason.

Project 3. Mini Quiz

What we study: program state, conditions, string functions, and blocking a repeated answer.

```
пусть очки = 0
пусть отвечено = ложь
окно "Викторина"
заголовок "Один вопрос"
текст "Какая планета известна как Красная планета?"
поле ответ "Введите название планеты"
текст результат "После ответа здесь появится результат."
текст "Очки: {очки}"
кнопка "Проверить ответ"
нажми:
  если отвечено
    сообщение "Вы уже ответили на этот вопрос."
  иначе если пусто(обрезать(ответ))
    сообщение "Введите ответ."
    фокус ответ
  иначе
    отвечено = истина
    если нижний(обрезать(ответ)) == "mars"
      очки += 1
      установить результат "Верно!"
    иначе
      установить результат "Не совсем. Правильный ответ: Марс."
    отключить ответ
```

Note: the comparison uses the string `"mars"` in lowercase because `нижний()` (lower) converts the text to lowercase. If you want to accept the Russian word “Марс” too, add a second comparison.

Check: the answers “Mars”, “mars”, and “MARS” are accepted, because `обрезать()` (trim) removes spaces and `нижний()` (lower) converts text to lowercase. A repeated click does not add a second point. The field is blocked after the answer.

Improve it yourself: add several questions; show the percentage of correct answers; implement a restart button that resets `очки` (score), `отвечено` (answered), and the answer field. When resetting, you must run `включить ответ` (enable) again.

Project 4. Countdown

What we study: state variables, a timer, input validation, and reactive display.

```
пусть осталось = 0
пусть идёт = ложь
окно "Обратный отсчёт"
заголовок "{осталось}"
  центр
  размер 48
поле длительность "Сколько секунд?"
текст статус "Готов к запуску"
  центр
строка
  кнопка "Старт"
  нажми:
    если идёт
      сообщение "Отсчёт уже идёт."
    иначе если не число_ли(длительность)
      сообщение "Введите количество секунд."
      фокус длительность
    иначе если число(длительность) < 1
      сообщение "Введите число больше нуля."
    иначе
      осталось = пол(число(длительность))
      идёт = истина
      установить статус "Отсчёт запущен"
  кнопка "Сброс"
  нажми:
    идёт = ложь
    осталось = 0
    установить статус "Сброшено"
каждую секунду:
  если идёт
    если осталось > 0
      осталось -= 1
    если осталось == 0
      идёт = ложь
      установить статус "Время вышло"
      сообщение "Отсчёт завершён!"
```

Check: enter 5, click “Start”, and watch the number decrease. Reset should stop the change, even though the timer keeps waking up and checking the flag `идёт` (running).

How it works: the timer starts once with the program. It does not create a new interval on each click; the variable `идёт` controls whether the counter changes. This approach is more reliable than starting a new timer on every click.

Improve it yourself: display the time in the format mm:ss using a function that adds a leading zero; add a pause; block the input field while the countdown is running.

CHAPTER 17

Quality Checks Before Publishing

A working program is not just an example that ran once. Check the typical scenarios, the extreme values, and the way you deliver the project.

17.1. Checklist

Before exporting, answer “yes” to every item that applies:

- [ ] The code runs without parse errors.
- [ ] Indentation shows the correct nesting, and every button has the right event.
- [ ] Names of fields, elements, variables, and functions are spelled identically everywhere.
- [ ] Empty input is handled explicitly.
- [ ] Numeric fields are checked with `число_ли()` (is_number) before conversion and calculation.
- [ ] Division by zero and an empty selection give a clear message.
- [ ] Lists are cleared before regenerating them, unless the old result must be kept.
- [ ] Reactive labels are not overwritten with `установить` (set) without need.
- [ ] Conditional properties are used only where they are supported.
- [ ] Timers are not created again on every user action.
- [ ] A network request has been tested with a real URL, and possible CORS problems are understood.
- [ ] The application has been tested after export as a separate HTML file.
- [ ] The .uis source file is saved as an editable copy.

17.2. Minimum Test Set for a Form

Scenario	Expected behaviour
All fields are empty	A clear prompt; no calculation is performed
Letters are entered in a numeric field	An error message, not a meaningless number
0 is entered	Handled according to the meaning of the field; not confused with an empty string
1.5 is entered	`число()` (number) converts the decimal comma
No option is selected	The program asks the user to make a choice
Checkbox is not ticked	The consent check fails
Click is repeated	No accidental doubling of points, list items, or timers
Button is pressed again after a reset	The application returns to a consistent initial state

17.3. Performance and Clarity

For small applications, UIscript does not require complex optimisation, but good habits still matter:

- Do not start many identical timers. Use one periodic timer and a state variable.
- Do not write excessively long loops: the environment has a limit on repetitions and protection against too many operations.
- Check input before expensive calculations and network requests.
- Do not repeat a large block of code several times if it can be moved into a function.
- Make messages specific: “Enter the second number” is better than “Error”.
- Use names such as `сумма` (sum), `длительность` (duration), and `результат` (result), not `x1`, `x2`, or `tmp`, if the names must still make sense in a week.

17.4. Limits of the Development Environment

For this version of the parser, keep in mind several specific limits:

- There are no `while`, `break`, or `continue` loops.
- A `повторить` (repeat) loop is limited to 10,000 passes.
- The depth of nested function calls is limited to 100.
- Up to 50 timers can be active at the same time.
- A single pass has protection against an excessive number of operations.
- Network requests have a timeout and a limit on parallel requests.
- Conditional properties are limited to `выключен` (disabled), `невидимый` (hidden), `жирный` (bold), `курсив` (italic), and `центр` (centre).

Note: these limits apply to the HTML implementation included with this book, not to every possible language called UIscript.

CHAPTER 18

Complete Quick Reference

This section is for quickly looking up syntax while you work.

18.1. Elements

Group	Commands
Basic	`окно` (window), `заголовок` (heading), `подзаголовок` (subheading), `текст` (text), `разделитель` (separator)
Actions	`кнопка` (button), `ссылка` (link)
Input	`поле` (field), `область` (text area), `галочка` (checkbox), `ползунок` (slider)
Lists	`выбор` (selection), `вариант` (option), `список` (list)
Media	`картинка` (image), `видео` (video)
Containers	`строка` (row), `столбец` (column), `карточка` (card)

18.2. Properties

Value type	Properties
Text, colour, address	`фон` (background), `цвет` (colour), `подсказка` (hint), `рамка` (border), `адрес` (address)
Numeric sizes	`размер` (size), `ширина` (width), `высота` (height), `отступ` (padding), `скругление` (rounding), `прозрачность` (opacity), `мин` (min), `макс` (max), `значение` (value)
Flags	`жирный` (bold), `курсив` (italic), `центр` (centre), `тень` (shadow), `невидимый` (hidden), `выключен` (disabled)
Conditional form	`жирный если …`, `курсив если …`, `центр если …`, `невидимый если …`, `выключен если …`

18.3. Events and Actions

Events: `нажми:` (click), `измени:` (change), `наведи:` (hover), `уход:` (leave).

Actions: `сообщение` (message), `показать` (show), `установить` (set), `очистить` (clear), `добавить` (add), `удалить` (remove), `скрыть` (hide), `открыть` (reveal), `переключить` (toggle), `включить` (enable), `отключить` (disable), `фокус` (focus), `запрос` (request), `вызвать` (call).

18.4. Control Structures

```
пусть имя = значение
имя = новое_значение
имя += 1

если условие
  действие
иначе если другое_условие
  действие
иначе
  действие

повторить 5 раз:
  показать "Проход {i} (индекс {индекс})"

функция квадрат(x):
  вернуть x * x

вызвать моя_функция(аргумент)

каждую секунду:
  действие

каждые 500 мс:
  действие

через 3 сек:
  действие
```

18.5. Operators

Category	Operators
Arithmetic	`+`, `-`, `*`, `/`, `%`
Comparison	`==`, `=`, `!=`, `>`, `<`, `>=`, `<=`
Logic	`и`, `или`, `не`; also `and`, `or`, `not`, `&&`, `` ` ``
Assignment	`=`, `+=`, `-=`, `*=`, `/=`, `%=`
Grouping	`(...)`

Use `==` in conditions and `=` for assignment, so that you do not confuse the operators' roles.

18.6. Built-in Functions

Function	Purpose	Example
`число(x)` (number)	Converts a string to a number; non-numeric input gives `NaN`	`число("1,5")` → 1.5
`число_ли(x)` (is_number)	Checks whether the value is a valid finite number	`число_ли("abc")` → false
`округлить(x, n)` (round)	Rounds to n decimal places; n ranges from 0 to 10	`округлить(2 / 3, 2)` → 0.67
`пол(x)` (floor)	Rounds down to an integer	`пол(3.7)` → 3
`потолок(x)` (ceiling)	Rounds up to an integer	`потолок(3.1)` → 4
`абс(x)` (absolute)	Absolute value	`абс(-5)` → 5

Function	Purpose	Example
`мин(a, b, …)` (min)	The smallest of the given numbers	`мин(3, 8)` → 3
`макс(a, b, …)` (max)	The largest of the given numbers	`макс(3, 8)` → 8
`случайное()` (random)	Random number from 0 inclusive to 1 exclusive	`случайное()`
`случайное(a, b)` (random)	Random integer from a to b inclusive	`случайное(1, 6)`
`длина(x)` (length)	Number of characters or array elements	`длина("привет")` → 6
`верхний(s)` (upper)	Converts text to uppercase	`верхний("abc")` → ABC
`нижний(s)` (lower)	Converts text to lowercase	`нижний("ДА")` → да
`обрезать(s)` (trim)	Removes spaces from both ends	`обрезать(" да ")` → да
`пусто(x)` (is_empty)	Checks for an empty value, string, `null`, `ложь`, or empty array	`пусто("")` → true
`путь(obj, "a.b[0]")` (path)	Reads a nested property of an object or array	`путь(ответ, "user.name")`
`сейчас()` (now)	Current device time in local format	`сейчас()`
`дата()` (date)	Current device date in local format	`дата()`
`секунд_до("ГГГГ-ММ-ДД ЧЧ:ММ")` (seconds_until)	Seconds until the given date and time; a negative number means the moment has passed	`секунд_до("2027-01-01 00:00")`

Note on `пусто()` (is_empty): spaces inside a string are not removed automatically. `пусто(" ")` is not the same as `пусто(обрезать(значение))` (is_empty(trim(value))) when you want to treat spaces alone as missing input.

18.7. Short Templates

The following examples are fragments, not standalone programs. Insert them into a suitable block of an existing application. For example, `добавить задачи ...` (add tasks) requires a previously created list `задачи` and usually goes inside a `нажми:` (click) handler. Field and variable names must match your own.

Number check:

```
если не число_ли(поле_числа)
  сообщение "Введите число."
иначе
  пусть x = число(поле_числа)
```

Short reactive label:

```
пусть счётчик = 0
текст "Счётчик: {счётчик}"
```

Adding to a list:

```
добавить задачи "{обрезать(новая_задача)}"
```

External request:

```
запрос "https://api.example.com/data" в ответ
```

Conditional display:

```
текст предупреждение "Проверьте данные"
  невидимый если не ошибка
```

CHAPTER 19

Exercises and Solutions

Try each exercise before reading the solution. You do not have to use exactly the same approach; what matters is that the program works correctly and is easy to understand.

Exercises

1. **Greeting.** Make a name field and a button. If the name is empty or contains only spaces, ask the user to enter a name; otherwise greet the person.
2. **Even or odd.** The user enters an integer. Show whether it is divisible by two without a remainder. Hint: use the remainder operator `%`.
3. **Task list.** Add a field and a button. An empty task must not be added. After adding, clear the field and show the number of tasks next to the list.
4. **Button generator.** Create five buttons with a loop. Each button must show its own number, not the number of the last button.
5. **Discount function.** Create a function that takes a price and a discount percentage and returns the final price. Test several examples.
6. **“Done” state.** Make a “Done” checkbox and a reactive label that changes depending on its value. Do not use a manual `установить` (set) for this reactive label.
7. **Timer.** Make a counter that increases once per second while the variable `идёт` (running) is `истина` (true). Add a “Start / Pause” button.
8. **API.** Make a request to a suitable public API, display the result, then extract one nested field from the JSON using `путь()` (path).

Possible Solutions

Solution 1. Greeting

```
окно "Приветствие"
поле имя "Введите имя"
кнопка "Поздороваться"
нажми:
  если пусто(обрезать(имя))
    сообщение "Введите имя."
    фокус имя
  иначе
    сообщение "Привет, {обрезать(имя)}!"
```

Solution 2. Even or odd

```
окно "Чётность"
поле число_ввода "Введите целое число"
кнопка "Проверить"
нажми:
  если не число_ли(число_ввода)
    сообщение "Введите число."
  иначе если число(число_ввода) % 2 == 0
    сообщение "Число чётное."
  иначе
    сообщение "Число нечётное."
```

Solution 3. Task list

```
окно "Задачи"
поле новая_задача "Текст задачи"
кнопка "Добавить"
нажми:
  если пусто(обрезать(новая_задача))
    сообщение "Введите задачу."
  иначе
    добавить задачи "{обрезать(новая_задача)}"
    очистить новая_задача
текст "Всего: {длина(задачи)}"
список задачи
```

Solution 4. Button generator

```
окно "Пять кнопок"
строка
  повторить 5 раз:
    кнопка "№ {i}"
      нажми:
        сообщение "Выбрана кнопка {i}"
```

Solution 5. Discount function

```
функция цена_со_скидкой(цена, процент):
  вернуть цена * (1 - процент / 100)

окно "Скидка"
текст "1000 со скидкой 15%: {округлить(цена_со_скидкой(1000, 15), 2)}"
```

Solution 6. Done or not done

```
окно "Статус"
галочка выполнено "Задача выполнена"
текст "Готово"
  невидимый если не выполнено
текст "Ещё в работе"
  невидимый если выполнено
```

Solution 7. Stopwatch

```
пусть секунды = 0
пусть идёт = ложь
окно "Секундомер"
заголовок "Время: {секунды} сек"
кнопка "Старт / Пауза"
нажми:
  идёт = не идёт
кнопка "Сброс"
нажми:
  секунды = 0
  идёт = ложь
каждую секунду:
  если идёт
    секунды += 1
```

Solution 8. API

Because the availability of public APIs and their CORS settings change over time, there is no universal URL that is guaranteed to work in every browser. Choose an API that allows browser requests, then use this template:

```
пусть ответ = null
окно "API"
кнопка "Загрузить"
нажми:
  запрос "https://ВАШ_API/путь" в ответ
текст "Ответ: {ответ}"
текст "Имя: {путь(ответ, 'user.name')}"
```

Replace the address with a real one, and replace `user.name` with the path to an existing field in the response. If the request fails, check the console, the HTTP status code, the CORS settings, and the data format. The URL in the example is intentionally not a working service.

CHAPTER 20

Conclusion

You have mastered the main elements and commands of UIscript: building an interface, variables and calculations, conditions, events, loops, functions, timers, reactivity, lists, and requests.

The next step is to keep building small, complete projects. Three simple applications where you understand every line are better than one huge example that is frightening to change.

A working order for a new project:

1. First describe the interface without any logic.
2. Give names to the elements whose values the program will need.
3. Add one event and make it work.
4. Connect variables, conditions, and input validation.
5. Add dynamic interface elements and any necessary timers.
6. Check the edge cases and export a standalone HTML file.
7. Keep the .uis source separate from the finished HTML.

The main goal is not to memorise every command, but to learn to break a task into small, clear steps and to check each one.

Compatibility note. This book targets the language behaviour of version 1.0, implemented in the file uis_1-0.html. If the implementation changes, check new command versions and limits against the corresponding HTML file of the development environment.